

Chapitre 1 : État de l'Art : LLMs Embarqués sur Smartphone

Introduction et contexte

L'essor des grands modèles de langage (LLMs) depuis la publication de GPT-3 par Brown et al. [1] a ouvert une problématique centrale : comment déployer ces systèmes aux capacités remarquables sur des dispositifs à ressources limitées ? Alors que GPT-3 mobilise 175 milliards de paramètres et nécessite plusieurs centaines de gigaoctets de mémoire GPU, la recherche s'est orientée vers des modèles compacts capables de s'exécuter directement sur smartphones, sans dépendance permanente au cloud, un paradigme désigné par l'expression *inférence embarquée (on-device inference)*.

Cette tendance répond à des enjeux concrets. Selon GSMA Intelligence [2], on comptait fin 2023 plus de 5,6 milliards d'abonnés mobiles uniques dans le monde (69 % de la population mondiale), chiffre le plus récent rapporté dans l'édition 2024 du *Mobile Economy Report*. Le smartphone est aujourd'hui le premier dispositif d'accès à l'information numérique, y compris dans des zones à connectivité intermittente. L'inférence de LLMs directement sur ces appareils est devenue techniquement viable depuis 2023 grâce à la convergence de trois facteurs : la miniaturisation des modèles (distillation, quantification), l'amélioration des SoCs mobiles (NPU/DSP), et le développement de frameworks d'inférence optimisés.

Concrètement, il s'agit de permettre des interactions en temps réel, sans connexion réseau et sans envoi de données à un serveur distant, sur des appareils dont la RAM dépasse rarement 12 Go et dont la puissance de calcul ne représente qu'une fraction de celle d'un GPU de datacenter. Ce chapitre présente les solutions disponibles en 2024-2026, compare leurs performances, et analyse les différences architecturales entre exécution locale et architectures hybrides edge + cloud. Ces éléments servent de base au protocole expérimental détaillé au chapitre 2.

1.1 Fondements théoriques : des grands modèles aux modèles embarqués

Avant de présenter les modèles et frameworks concrets, cette section pose les briques théoriques qui rendent l'inférence embarquée possible. Brown et al. [1] montrent avec GPT-3 que les capacités émergentes (few-shot, zero-shot) apparaissent brusquement à partir d'un certain seuil de paramètres, et non progressivement. Cela pose la question centrale de ce chapitre : comment rendre ces capacités accessibles sur un appareil de 6 Go de RAM ?

La quantification est la première réponse technique à cette question. Dettmers et al. [3] introduisent LLM.int8(), qui résout le problème des outliers par une décomposition mixte (0,1 % des poids en FP16, le reste en INT8) : division de la mémoire par deux avec moins de 1 % de dégradation. Frantar et al. [4] franchissent une étape supplémentaire avec GPTQ : descente à INT4 par minimisation d'erreur couche par couche, permettant à un LLaMA

7B de passer de 14 Go (FP16) à environ 4 Go (INT4). Le format GGUF [5] standardise la distribution de ces modèles quantifiés dans un fichier unique portable, avec plus de 100 000 fichiers publiés sur HuggingFace en 2024. Une synthèse récente [31] conclut que l'INT4 représente le meilleur compromis qualité/performance pour les modèles 3-7B sur flagship, avec une réduction documentée de 68,66 % pour LLaMA 3.2 3B après GPTQ INT4, un résultat cohérent avec le seuil critique de 3,5 bits par poids établi par la littérature plus récente [29] (section 1.5).

Le fine-tuning efficace constitue la deuxième brique. Dettmers et al. [6] introduisent QLoRA : quantification NF4 combinée à des adaptateurs LoRA bas-rang, permettant le fine-tuning d'un modèle 65B sur un seul GPU A100 48 Go. Google exploite directement ce principe dans ML Kit GenAI [12] via des adaptateurs LoRA spécialisés par tâche (résumé, relecture, réponse guidée). Enfin, la distillation de connaissance, posée par Hinton et al. [7], permet à un modèle étudiant d'imiter les soft targets d'un modèle enseignant plus grand. Cette technique est omniprésente dans les modèles mobiles actuels : LLaMA 3.2 1B/3B [16] sont distillés depuis LLaMA 3.1 8B, et Apple Intelligence [19] est distillé depuis un mélange d'experts (MoE) à 64 experts, pour une réduction de 90 % du coût d'entraînement. Ces quatre techniques (quantification, GGUF, QLoRA et distillation) forment la base théorique commune à tous les modèles présentés dans la suite de ce chapitre.

1.2 Panorama des modèles disponibles pour smartphone

Le paysage des modèles adaptés au smartphone s'est considérablement enrichi depuis 2023, porté par les principaux acteurs de l'IA ainsi que par des laboratoires de recherche spécialisés. L'histoire est récente mais dense : llama.cpp [8] tourne sur iPhone dès août 2023, Gemini Nano [9] est annoncé avec le Pixel 8 Pro en décembre 2023, Google publie Gemma [10] 2B/7B en février 2024, Microsoft publie Phi-3 Mini [18] en mars 2024, MLC-LLM [15] supporte les GPU Mali en juillet 2024, Meta publie LLaMA 3.2 [16] 1B et 3B en septembre 2024 comme premier Llama explicitement mobile-first, ML Kit GenAI [12] devient disponible en preview en octobre 2024, Gemini Nano 2 est déployé sur Pixel 9 et Galaxy S25 en janvier 2025, Google publie Gemma 4 en avril 2025, et ML Kit GenAI v1.0 devient stable en juin 2025.

1.2.1 Google : Gemini Nano, Flash/Flash-Lite et Gemma

Gemini Nano [9] est le modèle propriétaire de Google destiné à l'exécution on-device, décliné en Nano-1 (1,8 B, tâches de résumé et suggestion de réponse, déployé sur Pixel 8 Pro) et Nano-2 (~3,25 B, 79,6 % MMLU, surpassant plusieurs modèles deux fois plus grands, moteur de Pixel 9 et Galaxy S25/S26). Les deux variantes ne sont pas distribuées directement : elles sont accessibles uniquement via AICore [13], service système Android

qui gère le cycle de vie du modèle, l'accélération NPU et l'isolation de confidentialité, exposé aux développeurs via ML Kit GenAI [12] ou MediaPipe LLM Inference API [14]. Distinction essentielle pour ce PIR : contrairement à Gemini Nano, Gemini Flash et Flash-Lite [11] ne s'exécutent pas localement sur le smartphone. Ce sont des modèles cloud optimisés pour la faible latence, accessibles via l'API Gemini depuis une application mobile. Ils s'intègrent typiquement dans une architecture hybride où l'application décide, selon la complexité de la tâche, de solliciter Gemini Nano on-device ou Gemini Flash via l'API cloud (section 1.6). Gemma [10], famille open source de Google disponible en GGUF pour llama.cpp ou via LiteRT, comprend Gemma 2 2B IT (2,6 B, ~1,5 Go, 51,3 % MMLU), Gemma 2 9B IT (9 B, ~5,5 Go, 71,3 %), Gemma 3 4B IT (4 B, contexte 32 768, 59,6 %) et les variantes Edge de Gemma 4 (E2B et E4B, ~1,2 et ~2,4 Go), spécifiquement optimisées pour les NPU mobiles via le format LiteRT.

1.2.2 Meta : LLaMA 3.2

LLaMA 3.2 1B Instruct (1,24 B, ~771 Mo, contexte théorique 128 000, 32,2 % MMLU) et 3B Instruct (3,21 B, ~2,0 Go, 58,0 % MMLU) [16] sont les seules versions réalistes pour un déploiement smartphone. La fenêtre de contexte théorique de 128 000 tokens est irréaliste en pratique mobile faute de RAM suffisante ; les contextes pratiques sont de 2 048 à 4 096 tokens. LLaMA 3.2 1B est le modèle standard retenu pour les benchmarks du chapitre 2, en tant que référence minimale pour un déploiement ARM64.

1.2.3 Microsoft : Phi-3/Phi-4 Mini

Microsoft a adopté une approche « small but capable » avec la série Phi [18] : Phi-3 Mini 4K et 128K (3,8 B, ~2,3 Go, 68,8 % MMLU), Phi-3.5 Mini (69,0 %) et Phi-4 Mini (72,8 %). Abdin et al. montrent que Phi-3 Mini dépasse Mistral 7B sur MMLU (68,8 % vs 61,7 %) et atteint 82,5 % sur GSM8K grâce à des données synthétiques distillées depuis GPT-4, le meilleur score de raisonnement arithmétique parmi les modèles inférieurs à 4B (section 1.3). Cette famille se distingue par un score très élevé pour sa taille, au prix d'un fort biais anglophone et d'une performance dégradée en français.

1.2.4 Apple et directions de recherche complémentaires

Apple a adopté une approche radicalement fermée avec Apple Intelligence [19] (iOS 18+) : un modèle d'environ 3B de paramètres compressé à 2 bits/poids, exécuté via le Neural Engine, avec une technique de KV Cache Sharing réduisant de 37,5 % la mémoire et le temps jusqu'au premier token. Ces modèles sont inaccessibles sur Android et ne sont pas traités en détail dans ce PIR, mais servent de point de comparaison pour situer l'écosystème Android (section 1.6). Deux directions de recherche complémentaires méritent mention : MobileLLM [17] (Meta Research), qui montre qu'à l'échelle sub-

milliard l'architecture prime sur les données : un modèle profond et fin surpasse systématiquement un modèle large et plat de même taille, avec des performances proches de LLaMA 2 7B. PowerInfer-2 [20] exploite quant à lui la sparsité des activations (30 % de neurones actifs par token) en streamant les poids inactifs depuis le stockage flash. Il exécute ainsi TurboSparse-Mixtral 47B sur smartphone à 11,68 tokens/s, soit 22× plus vite que llama.cpp.

Modèle	Éd.	Params	Taille Q4	RAM min	MMLU	Français	Exécution
LLaMA 3.2 1B	Meta	1,2 B	771 Mo	3 Go	32 %	Moyen	On-device
LLaMA 3.2 3B	Meta	3,2 B	2,0 Go	5 Go	58 %	Bon	On-device
Gemma 2 2B	Google	2,6 B	1,5 Go	4 Go	51 %	Moyen	On-device
Gemma 4 E2B	Google	2 B	1,2 Go	3 Go	56 %	Bon	On-device
Phi-4 Mini	Microsoft	3,8 B	2,3 Go	5 Go	73 %	Faible	On-device (MIT)
Qwen 2.5 1.5B	Alibaba	1,5 B	900 Mo	3 Go	46 %	Très bon	On-device
Gemini Nano 2	Google	~3,25 B	N/A (AICore)	Géré AICore	79,6 %	Bon	On-device (AICore)
Gemini Flash	Google	—	—	N/A	—	Excellent	Hybride (API cloud)
MobileLLM 1B	Meta	<1 B	~500 Mo	~0,9 Go	~52 %	Faible	On-device

Tableau 1.1 : Comparaison des modèles LLM disponibles pour smartphone

1.3 Comparaison des capacités qualitatives

Au-delà du seul score MMLU, les capacités pratiques des modèles divergent selon quatre axes : raisonnement, instruction-following, support multilingue et tâches pratiques mobiles, des critères aussi déterminants que la taille ou la vitesse brute pour choisir un modèle en contexte réel. Sur le raisonnement et l'arithmétique, le benchmark GSM8K est un indicateur robuste : Phi-3 Mini domine avec 82,5 %, LLaMA 3.2 3B atteint environ 58 %, Gemma 2 2B environ 46 %, tandis que les modèles sub-milliard comme MobileLLM montrent des limitations importantes sur les tâches multi-étapes (moins de 20 %). Aucun modèle inférieur à 4B ne dépasse 70 % sur ARC-Challenge, ce qui situe clairement le plafond de qualité des solutions on-device face au cloud.

Sur l'instruction-following, Gemini Nano-2 excelle en résumé, reformulation et complétion guidée grâce à ses adaptateurs QLoRA par tâche ; Phi-3 Mini présente le meilleur instruction-following général parmi les modèles open source inférieurs à 4B ; LLaMA 3.2 3B offre de bonnes performances en conversation mais décroche sur les instructions

complexes à contraintes multiples ; Gemma 2 2B est particulièrement adapté à la classification et l'extraction d'entités. Sur le support multilingue, Gemini Nano bénéficie d'un support robuste natif avec des performances en français proches de l'anglais ; Gemma 2 2B couvre correctement les 35 langues du préentraînement Gemini ; Phi-3 Mini souffre d'un préentraînement orienté anglais (15 à 20 % de baisse en français) ; LLaMA 3.2 se situe entre les deux avec un biais anglophone marqué.

Modèle	Raisonnement	Instruct.	Multilingue	Tâches mobiles
Gemini Nano-2	Moyen	Excellent	Excellent	Résumé, relecture
Phi-3 Mini	Excellent	Très bon	Moyen	QA, code
LLaMA 3.2 3B	Bon	Bon	Moyen	Chat, QA
Gemma 2 2B	Moyen	Bon	Bon	Classification, NER
MobileLLM 1B	Faible	Moyen	Faible	Suggestion, auto-complétion
Gemini Flash	Élevé	Excellent	Excellent	Toutes (hybride)

Tableau 1.2 : Comparaison des capacités qualitatives par modèle

Enfin, quatre catégories de tâches structurent l'essentiel des cas d'usage on-device : résumé de contenu, suggestion de réponse, classification locale et extraction d'information. Chacune est contrainte différemment selon l'API level Android et le SoC ciblé : ML Kit GenAI/AICore exige Android 10+ et un appareil certifié ; llama.cpp via Termux est compatible Android 7+ sur tout ARM64 sans restriction matérielle ; MLC-LLM cible Android 8+ avec GPU Vulkan recommandé ; MediaPipe LLM API cible Android 10+ avec délégué GPU.

1.4 Contraintes matérielles des smartphones

L'exécution d'un LLM sur smartphone est soumise à des contraintes radicalement différentes du serveur. Le Snapdragon 8 Gen 3 [21], pris comme référence haut de gamme 2024, illustre la répartition typique d'un SoC mobile moderne : CPU Cortex-X4 à 3,3 GHz, GPU Adreno 750 (4,7 TFLOPS FP16), NPU Hexagon (98 TOPS INT8) et 16 Go de LPDDR5X. Le NPU Hexagon accélère le prefill d'un facteur 50× par rapport au CPU sur LLaMA 2 7B [22]. Cet écart explique pourquoi les solutions exploitant le NPU (ML Kit GenAI, LiteRT) surclassent largement le CPU pur (llama.cpp) sur les appareils compatibles.

Côté mémoire, Android consomme à lui seul 4 à 6 Go sur les 12-16 Go disponibles d'un appareil moderne. La phase de decode étant memory-bound, la bande passante mémoire prime sur la puissance de calcul brute : un Snapdragon 8 Gen 3 offre environ 77 Go/s en LPDDR5X contre environ 34 Go/s pour un Exynos 1380 en LPDDR4X [22], ce qui explique l'essentiel de l'écart de performances observé entre familles de SoC, un écart que confirment les mesures du chapitre 2 entre Snapdragon et Exynos. La fenêtre de contexte consomme également de la RAM proportionnellement à sa taille via le KV cache : 2 048 tokens représentent environ 500 Mo supplémentaires pour un modèle 2B, 8 192

tokens environ 2 Go supplémentaires, ce qui devient critique sur un appareil à 6 Go de RAM totale.

Enfin, la consommation énergétique et le throttling thermique sont la principale source d'instabilité. La littérature [22, 23] rapporte une consommation de 5 à 10 % de batterie par 10 minutes de génération intensive ; les NPU réduisent cette consommation de 3 à 5× par rapport au GPU. Le throttling thermique dégrade les performances de 10 à 20 % en session longue, et jusqu'à 15 à 25 % sur les appareils milieu de gamme. Ces ordres de grandeur servent de référence directe pour les mesures de terrain présentées au chapitre 2, qui montrent des chutes de performance globalement inférieures sur le modèle 1B testé, à l'exception de deux appareils spécifiques.

1.5 Frameworks d'inférence mobile

llama.cpp [8], développé par Georgi Gerganov en 2023, est devenu le standard de facto pour l'inférence de LLMs quantifiés sur CPU, avec une implémentation C/C++ zéro-dépendance exploitant ARM NEON et AVX2 et le format GGUF [5] natif. Il est compatible avec tout ARM64 Android sans NPU requis, fonctionne via Termux sans root ni Android Studio, mais reste CPU-only sur la grande majorité des appareils Android faute de support GPU Mali. C'est le framework de référence retenu pour le benchmark de ce PIR, tous appareils confondus.

ML Kit GenAI (AICore) [12, 13], introduit par Google en 2024, est l'API officielle pour accéder à Gemini Nano via l'AICore d'Android, exposant des cas d'usage intégrés (résumé, relecture, réécriture, description d'image) via des adaptateurs QLoRA [6] par tâche. Elle offre un accès au NPU pour des performances maximales et une API simple en Kotlin/Java, mais reste limitée aux appareils certifiés (Pixel 9/10, Galaxy S25/S26), excluant environ 95 % du parc Android mondial, avec un modèle non modifiable et un quota d'inférence par application. C'est la solution Google propriétaire testée sur Galaxy S26 dans ce PIR.

LiteRT, anciennement TensorFlow Lite, est le runtime d'inférence Google pour modèles .tflite ; en 2024, Google a migré TFLite vers LiteRT et ajouté le support LLM via le framework AI Edge LLM Inference. Ce framework supporte NPU et GPU et est utilisé par Google AI Edge Gallery pour Gemma 4 Edge. Il est plus portable que ML Kit GenAI, mais nécessite un format de conversion non trivial depuis GGUF. MediaPipe LLM Inference API [14] offre une alternative plus flexible reposant sur le même runtime : elle supporte Gemma, Phi-2 et Falcon 1B via un pipeline unifié, sans exiger d'appareil certifié. MLC-LLM [15], du groupe MLC AI, utilise Apache TVM pour compiler des modèles directement en code GPU/NPU optimisé ; c'est le seul framework open source à exploiter réellement les GPU Mali via Vulkan, 20 à 25 % plus rapide que llama.cpp sur Snapdragon 8 Gen 3

et Dimensity 9300, au prix d'une compilation par cible matérielle et d'un écosystème de modèles plus restreint que GGUF.

Critère	llama.cpp	MLC-LLM	ML Kit GenAI	MediaPipe LLM
Modèles	GGUF universel	TVM compilé	Gemini Nano seul	Gemma, Phi-2, Falcon
GPU Mali	Non	Oui (Vulkan)	NPU AICore	Oui (Vulkan/OpenCL)
Android min.	7+	8+	10+ certifié	10+
Appareils	Tout ARM64	Tout (Vulkan rec.)	Flagships certifiés	Tout (GPU)
Perf. relative	1×	~1,2-1,25×	~1,5-2×	~1,0-1,1×
Open source	Oui (MIT)	Oui (Apache 2)	Non	Oui (Apache 2)

Tableau 1.3 : Comparaison des frameworks d'inférence mobile

Au-delà des caractéristiques techniques, le portage effectif se heurte à des écarts matériels significatifs : Fassold [23] confirme un écart de performance iOS/Android de 3× (Apple Neural Engine contre GPU OpenCL) et un throttling thermique de 15 à 25 % sur le milieu de gamme après seulement 5 minutes d'utilisation. Ces chiffres se comparent directement aux mesures obtenues en conditions réelles au chapitre 2, qui confirment partiellement cette fourchette sur les appareils Exynos d'entrée de gamme testés.

1.6 Exécution locale pure vs architecture hybride (edge + cloud)

L'exécution entièrement locale signifie que le modèle tourne sur l'appareil sans aucun appel réseau. Elle garantit une vie privée maximale, une disponibilité hors-ligne totale, une latence déterministe et un coût opérationnel nul, mais souffre d'une qualité limitée aux modèles 1-7B en pratique, insuffisante pour le raisonnement multi-étapes complexe face à GPT-4o ou Gemini Flash, d'un contexte court (2 048-4 096 tokens en pratique), d'une dégradation thermique sur sessions longues et d'un espace de stockage requis de 1 à 5 Go par modèle téléchargé. Ses cas d'usage idéaux restent le clavier prédictif, les suggestions en temps réel, le résumé de notes, l'extraction d'entités et le chatbot FAQ hors-ligne. Deux directions de recherche visent à dépasser ces limitations sans quitter le paradigme local : Yin et al. [25] proposent le LLM comme service partagé au niveau du système d'exploitation, avec une réduction de 72 % de la RAM par mutualisation entre applications ; Ye et al. [26], avec `prima.cpp`, démontrent une inférence distribuée sur un cluster Wi-Fi domestique atteignant 674 ms/token pour un modèle 70B avec moins de 6 % de RAM consommée par appareil.

L'architecture hybride combine un modèle local léger et un service cloud plus puissant, avec un routeur qui décide où exécuter chaque requête selon sa complexité. C'est

précisément le positionnement de Gemini Flash et Flash-Lite [11] face à Gemini Nano (section 1.2). Le schéma ci-dessous illustre ce mécanisme de routage :

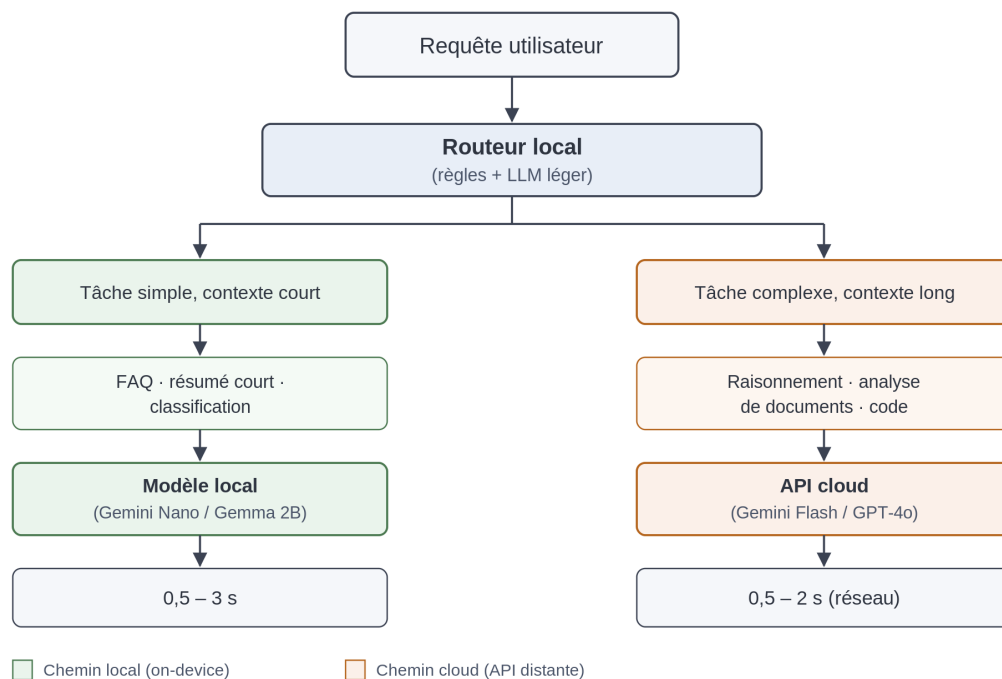


Figure 1.1 : Mécanisme de routage d’une architecture hybride edge + cloud.

Critère	Local pur	Hybride	Cloud pur
Confidentialité	Totale	Partielle	Nulle
Qualité de réponse	Limitée (1-7B)	Élevée	Maximale
Disponibilité hors-ligne	Totale	Dégradée	Nulle
Latence	Faible	Variable	Réseau
Coût opérationnel	Nul	Modéré	Élevé

Tableau 1.4 : Comparaison des architectures d’exécution locale, hybride et cloud

Apple pousse cette logique plus loin avec Private Cloud Compute (PCC) : les requêtes dépassant le modèle embarqué sont acheminées vers des serveurs Apple Silicon offrant des garanties cryptographiquement vérifiables, données supprimées immédiatement après traitement et inaccessibles même au personnel Apple. Ce modèle est aujourd’hui une référence en matière d’architecture hybride respectueuse de la vie privée. Cinq positionnements coexistent dans ce PIR : llama.cpp + LLaMA 3.2 1B (100 % local, benchmarking et R&D), ML Kit GenAI/Gemini Nano (100 % local via AICore, apps grand public sur flagship), Google AI Edge Gallery (100 % local via LiteRT, démonstration Gemma Edge), Gemini Flash API (100 % cloud, référence qualité) et l’architecture hybride (local + cloud, production scalable).

1.7 Critères de choix d'une solution

Le choix d'un framework et d'un modèle dépend de plusieurs critères interdépendants. Sur le parc cible, ML Kit GenAI est idéal si l'on cible uniquement les flagships 2024+ (Pixel 9/10, Galaxy S25/S26) ; pour tout autre appareil Android, llama.cpp reste la seule option CPU universelle. Sur le contrôle du modèle, si le PIR ou l'application requiert un modèle fine-tuné ou personnalisé, ML Kit GenAI est exclu d'office car le modèle est figé, alors que llama.cpp ou MLC-LLM permettent de charger n'importe quel GGUF. Sur les performances, ML Kit GenAI via NPU surpasse llama.cpp sur les appareils compatibles, mais MLC-LLM via Vulkan offre les meilleures performances sur les appareils non compatibles AICore. Sur la vie privée, les deux solutions restent locales, bien que ML Kit GenAI puisse potentiellement transmettre des métadonnées à Google. Sur la reproductibilité enfin, llama.cpp est entièrement open source et reproductible sur n'importe quelle machine, tandis que ML Kit GenAI dépend d'un écosystème Google propriétaire susceptible d'évoluer sans préavis. Cette tension entre flexibilité universelle et performance optimisée sur un parc restreint justifie l'étude conjointe des deux solutions dans ce PIR.

1.8 Études de performance publiées et positionnement du PIR

Plusieurs travaux récents mesurent les performances de LLMs sur appareils réels et forment un cadre de comparaison direct pour les résultats de ce PIR. Xu et al. [22] couvrent 11 appareils COTS avec llama.cpp et MLC-LLM : sur Snapdragon 8 Gen 3, le prefill atteint 30 à 56 tokens/s en INT8. MLC-LLM se situe 20 à 25 % au-dessus de llama.cpp, avec un throttling thermique de 10 à 20 % en session longue. PalmBench [27] propose un benchmark systématique de LLMs compressés sur plateformes mobiles avec llama.cpp, couvrant latence CPU/GPU, consommation énergétique et empreinte mémoire. C'est la méthodologie la plus proche du protocole adopté dans ce PIR, ici étendue aux appareils Samsung Exynos entrée/milieu de gamme absents de PalmBench. MobileAIBench [28] évalue llama.cpp sur appareils réels avec 20 datasets couvrant NLP, tâches multimodales et sécurité ; il complète l'approche de ce PIR, centrée sur les métriques d'exécution.

Song et al. [29] évaluent 7 méthodes PTQ sur des modèles de 0,5B à 14B paramètres et établissent un seuil critique à 3,5 bits par poids (BPW) : en dessous de cette densité, la qualité chute significativement, et un grand modèle quantifié Q4 dépasse même un petit modèle FP16 de taille inférieure. Le format Q4_K_M (~4,5 BPW) utilisé dans ce PIR se situe au-dessus de ce seuil critique, ce qui valide indépendamment le choix de quantification retenu. Tummalapalli et al. [30] mesurent un Galaxy S24 Ultra et un iPhone 16 Pro sous charge soutenue avec Qwen 2.5 1.5B Q4 : le GPU du S24 Ultra subit un arrêt complet de l'inférence par throttling thermique lors des sessions prolongées, un

phénomène qui converge avec les mesures sur Galaxy S26 du chapitre 2 (−17,3 % de dégradation thermique confirmée). PowerInfer-2 [20] et sa décomposition en clusters de neurones, avec $29,2\times$ d'accélération théorique, représentent une direction de recherche avancée non encore reproduite de manière indépendante.

1.9 Tendances et perspectives (2025-2027)

Plusieurs évolutions se dessinent pour l'écosystème mobile d'ici 2027. La compression agressive se poursuit vers la quantification INT2 (2 bits par paramètre) : des travaux comme QuaRot, GPTQ [4] et BitNet.cpp suggèrent que des modèles 7B en INT2 ($\sim 3,5$ Go) deviendront viables sur les appareils haut de gamme dès 2026, tandis que QLoRA [6] reste la référence pour personnaliser ces modèles compressés après coup. La tendance lourde est par ailleurs le passage du CPU au NPU comme cible d'inférence principale, avec des performances 5 à $10\times$ supérieures au CPU à consommation équivalente. Le décodage spéculatif, où un modèle local léger génère des tokens candidats vérifiés par un modèle plus grand, réduit la latence de 2 à $3\times$ dans les configurations hybrides ; prima.cpp [26] atteint ainsi 26 tokens/s pour un modèle 32B en configuration distribuée.

Des travaux académiques (Yin et al. [25]) proposent par ailleurs de déporter le modèle LLM au niveau du système d'exploitation, partagé entre applications comme un service OS. Cette direction est déjà amorcée par AICore [13] côté Android et par Apple Intelligence [19] côté iOS ; elle éliminerait la duplication des modèles en mémoire. Les modèles multimodaux progressent également : Gemini Nano 2 Multimodal [9] et Apple Intelligence supportent déjà l'analyse d'images localement, et Gemma 3 introduit le support vision natif pour ses variantes 4B et 12B. Enfin, l'architecture agentique on-device (ReAct, Tool-calling) reste limitée par les capacités de raisonnement des petits modèles, mais progresse via le protocole MCP et les recherches sur les Small Action Models. Des applications concrètes sont attendues pour 2026-2027.

1.10 Synthèse et conclusion

Axe	Situation 2025-2026	Horizon 2027
Modèles disponibles	1-4B paramètres viables	7-13B viables (INT2)
Frameworks matures	llama.cpp (CPU), ML Kit (NPU)	MLC-LLM + LiteRT convergence
Parc compatible NPU	Flagships uniquement ($\sim 5\%$)	Haut/milieu de gamme
Qualité vs cloud	Écart MMLU de 15-30 pts	Écart de 5-15 pts
Cas d'usage matures	FAQ, résumé, classification	Agents, RAG local
Thermique	Problème non résolu	Amélioration partielle (3nm+)

Tableau 1.5 : Synthèse de l'état de l'art et perspectives 2027

L'exécution de LLMs sur smartphone est passée, en moins de trois ans, du statut de curiosité technique à celui de réalité déployée sur des centaines de millions d'appareils via Galaxy AI et Apple Intelligence. Les avancées en quantification, en distillation et en architectures compactes ont rendu l'exécution locale viable ; les SoCs modernes offrent une puissance suffisante pour des modèles de 1 à 4B paramètres, confirmée par la littérature comme par les mesures propres de ce PIR.

Deux approches structurent aujourd'hui ce paysage. D'un côté, l'approche propriétaire (Google AICore et Gemini Nano, Apple Intelligence) exploite des NPU dédiés, au prix d'une dépendance matérielle stricte limitée à environ 5 % du parc Android mondial. De l'autre, l'écosystème open source (llama.cpp, MLC-LLM, Gemma) offre une flexibilité maximale sur tout appareil ARM64, mais avec des performances brutes inférieures sur les appareils compatibles NPU. Gemini Flash et Flash-Lite couvrent le segment hybride à haute capacité : une troisième voie où le choix ne se pose plus entre local et cloud, mais dans l'orchestration des deux. Cette tension entre flexibilité universelle et performance optimisée sur un parc restreint, ainsi que la troisième voie hybride, oriente les choix méthodologiques du chapitre suivant, consacré au déploiement effectif et à la mesure des performances sur des appareils réels.